ROOTCAUSE SERVER

# 02 · Instalación y ejecución

**Documento:** 02-installation-and-execution.md **Versión analizada:** 0.2.0 **Commit:** 2efedb2 **Fecha del análisis:** 2026-08-26

CONTENIDO


- Requisitos previos
- Obtener el código
- Instalación de dependencias
- Compilación
- Variables de entorno
- Configuración inicial
  - 1. Generar un token
  - 2. Preparar la base de datos
  - 3. Revisar la política de detección
- Ejecución en desarrollo
  - Camino corto
  - Camino explícito
  - Modo sin token
  - Enrolar el primer servidor
- Ejecución en producción
  - Con los binarios y systemd (Linux)
  - Con Docker Compose
  - Detrás de un proxy inverso
- Ejecución de pruebas
- Puerta de finalización
- Verificación de que todo funciona
- Errores frecuentes durante la instalación
- Errores frecuentes de la primera ejecución


> Todo lo necesario para pasar de un repositorio recién clonado a un plano de control funcionando, con su primer servidor enrolado. Los comandos son verificables y ningún valor sensible de este documento es real.

## Requisitos previos

| Requisito | Versión | Cómo lo fija el repositorio |
|---|---|---|
| Rust | `1.95.0` exacto | `rust-toolchain.toml` lo instala automáticamente vía `rustup` |
| Componentes de Rust | `clippy`, `rustfmt` | Declarados en el mismo archivo, perfil `minimal` |
| Enlazador del sistema | El del sistema operativo | Necesario para compilar cualquier binario Rust |

| Requisito | Versión | Cómo lo fija el repositorio |
|---|---|---|
| Python | 3.x | Solo para los guardianes de CI y `scripts/sbom.py` |
| Bash | — | Solo para `scripts/smoke.sh`, `scripts/dev.sh` y `scripts/run-agent.sh` |
| Docker + Compose | — | Opcional, solo para el despliegue en contenedor |
| Sistema operativo | Windows 10/11 o Server, Linux con systemd, o macOS | Las tres plataformas están cubiertas por la matriz de CI |

No hace falta instalar SQLite: `sqlx` lo lleva embebido y el archivo de base de datos se crea solo (`create_if_missing(true)` en `storage.rs`).

> **Nota de entorno local.** En la máquina Windows donde se hizo este análisis, la compilación de Rust se ejecuta dentro de WSL porque el toolchain de Windows no tiene un enlazador utilizable. Es una particularidad de esa máquina, no un requisito del proyecto: la matriz de CI compila y prueba en `windows-latest` de forma nativa.

## Obtener el código

```
git clone https://github.com/vladimiracunadev-create/rootcause-server.git
```

```
cd rootcause-server
```

## Instalación de dependencias

No hay paso de instalación separado: `cargo` resuelve el árbol al compilar, y `Cargo.lock` está versionado, así que el resultado es reproducible.

Para descargar las dependencias sin compilar:

```
cargo fetch --locked
```

## Compilación

Compilación de desarrollo, con todo el workspace:

```
cargo build --workspace --locked
```

Compilación de publicación (LTO fino, un solo `codegen-unit`, símbolos eliminados, `panic = abort`):

```
cargo build --release --locked
```

Perfil `dist`, que hereda de `release` y añade LTO completo. Es el que usa el workflow de publicación:

```
cargo build --profile dist --locked -p rootcause-server
```

Los binarios quedan en `target/debug/`, `target/release/` o `target/dist/` según el perfil.

## Variables de entorno

Cada opción de línea de comandos tiene su variable equivalente. El archivo `.env.example` las lista todas con comentarios. Las imprescindibles:

| Variable | Valor por omisión | Para qué |
|---|---|---|
| `ROOTCAUSE_API_TOKEN` | *(sin valor)* | Token compartido. **Obligatorio** salvo en modo de desarrollo inseguro |
| `ROOTCAUSE_BIND` | `127.0.0.1:8080` | Dirección de escucha del servidor |
| `ROOTCAUSE_DATABASE_URL` | `sqlite://rootcause.db` | Ubicación del archivo SQLite |
| `ROOTCAUSE_SERVER_URL` | `http://127.0.0.1:8080` | URL que usa el agente |
| `ROOTCAUSE_LOG` | `rootcause_server=info,tower_http=warn` | Filtro de registros, sintaxis `EnvFilter` |

El catálogo completo, con rangos válidos y consecuencias de equivocarse, está en 10 · Configuración.

Punto de partida recomendado:

```
cp .env.example .env
```

El archivo `.env` **no se carga automáticamente**: el binario no lee archivos `.env`. Sirve como plantilla para tu gestor de servicios o para exportar las variables a mano.

## Configuración inicial

### 1. Generar un token

```
cargo run -p rootcause-server -- token
```

Imprime 64 caracteres hexadecimales (dos UUID v4 concatenados en forma `simple`). El servidor rechaza cualquier token de menos de 32 caracteres, y también uno con espacios al principio o al final.

> **No pases el token por la línea de comandos.** Queda en el historial del shell y es visible en `ps`. Úsalo siempre como variable de entorno, tal como advierte `.env.example`.

### 2. Preparar la base de datos

No hay ningún paso manual. Al arrancar, `Database::connect` crea el archivo si falta, activa claves foráneas, fija un tiempo de espera de 10 segundos, usa diario WAL (salvo en base de datos en memoria) y ejecuta las migraciones pendientes de `crates/rootcause-server/migrations/`.

Para restaurar un despliegue existente basta con colocar el archivo `.db` en la ruta que apunte `ROOTCAUSE_DATABASE_URL`. Ver 07 · Base de datos.

### 3. Revisar la política de detección

Imprimir la política vigente, lista para versionar junto al despliegue:

```
cargo run -p rootcause-server -- policy
```

Validar una política propia **antes** de usarla:

```
cargo run -p rootcause-server -- policy --file politica.json
```

Si la política es contradictoria (por ejemplo `cpu_high` mayor que `cpu_critical`), el comando falla y el servidor se niega a arrancar con ella.

## Ejecución en desarrollo

### Camino corto

```
bash scripts/dev.sh
```

Genera un token si no hay uno en el entorno, lo imprime y arranca el servidor. En PowerShell, el equivalente:

```
pwsh -File scripts/dev.ps1
```

### Camino explícito

En Bash:

```
export ROOTCAUSE_API_TOKEN="pega-aqui-el-token"
```

```
cargo run -p rootcause-server -- serve
```

En PowerShell:

```
$env:ROOTCAUSE_API_TOKEN = "pega-aqui-el-token"
```

```
cargo run -p rootcause-server -- serve
```

La consola queda en `http://127.0.0.1:8080`. Pide el token la primera vez y lo guarda en `sessionStorage` del navegador.

### Modo sin token

```
cargo run -p rootcause-server -- serve --insecure-dev-mode
```

Solo funciona escuchando en loopback: `ServeSettings::validate` rechaza la combinación de modo inseguro con una dirección que no sea de loopback. El servidor lo advierte por registro al arrancar y la consola muestra un aviso permanente, porque `GET /api/v1/status` devuelve `hardening.authentication: false`.

### Enrolar el primer servidor

Antes de enviar nada, ver **exactamente** qué se enviaría:

```
cargo run -p rootcause-agent -- --dry-run
```

Un solo ciclo, declarando el rol y el entorno del equipo:

```
cargo run -p rootcause-agent -- --once --label role=edge --label environment=production
```

Reportando de forma continua cada 30 segundos (quitando `--once`):

```
cargo run -p rootcause-agent -- --label role=database
```

El rol importa: cambia qué se considera normal. Un servidor de borde *puede* publicar 80 y 443 sin generar hallazgo; un servidor de base de datos alcanzable públicamente es crítico siempre.

Variantes útiles del agente:

```
cargo run -p rootcause-agent -- --metrics-only
```

```
cargo run -p rootcause-agent -- --watch-file /srv/app/.env --watch-file /etc/nginx/nginx.conf
```

## Ejecución en producción

### Con los binarios y systemd (Linux)

Las unidades ya están en el repositorio:

- `packaging/linux/rootcause-server.service`
- `packaging/linux/rootcause-agent.service`

Ambas corren como usuario dedicado, con `NoNewPrivileges`, `ProtectSystem=strict`, `CapabilityBoundingSet` vacío, `MemoryDenyWriteExecute` y familias de sockets restringidas. Leen su configuración de `/etc/rootcause/server.env` y `/etc/rootcause/agent.env`.

### Con Docker Compose

```
export ROOTCAUSE_API_TOKEN="pega-aqui-el-token"
```

```
docker compose up -d --build
```

`compose.yml` publica el puerto **solo en el loopback del anfitrión** (`127.0.0.1:8080:8080`), monta el sistema de archivos como solo lectura, descarta todas las capacidades, impide elevar privilegios y limita la rotación de registros. La comprobación de salud usa `/dev/tcp` desde `bash` en lugar de añadir `curl` a una imagen que no lo necesita.

Dentro del contenedor, `ROOTCAUSE_BIND` vale `0.0.0.0:8080` porque, dentro de un contenedor, escuchar en loopback significa ser inalcanzable. Lo que compensa esa apertura es que el puerto solo se publica en `127.0.0.1` del anfitrión.

### Detrás de un proxy inverso

El orden importa y está documentado en el `README.md`:

1. Poner Caddy, Nginx, Traefik o un balanceador **con TLS** delante.
2. Cambiar `ROOTCAUSE_BIND` **solo después** de proteger la ruta.
3. Activar `--trust-forwarded-for` **únicamente** si hay proxy inverso. El servidor rechaza esa opción si detecta que escucha en loopback, porque sin proxy cualquiera elegiría qué dirección se limita y se bloquea.
4. Instalar el agente con una URL `https://`. Con una URL `http://` remota el agente se niega a enviar el token salvo `--allow-insecure-http` explícito.

## Ejecución de pruebas

Suite completa del workspace:

```
cargo test --workspace --all-targets --locked
```

Pruebas de documentación:

```
cargo test --workspace --doc --locked
```

Prueba de humo extremo a extremo: levanta el servidor y el agente reales y comprueba lo que comprobaría un operador a mano.

```
bash scripts/smoke.sh
```

Puerto configurable con `ROOTCAUSE_SMOKE_PORT` (por omisión `18080`).

Los tres guardianes propios del repositorio:

```
python3 scripts/guard_console.py
```

```
python3 scripts/guard_claims.py
```

```
python3 scripts/guard_encoding.py
```

Formato y análisis estático:

```
cargo fmt --all -- --check
```

```
cargo clippy --workspace --all-targets -- -D warnings
```

## Puerta de finalización

`AGENTS.md` define la secuencia que debe pasar antes de dar un cambio por terminado:

```
cargo fmt --all -- --check && cargo clippy --workspace --all-targets -- -D warnings && cargo test --workspace && bash scripts/smoke.sh && python3 scripts/guard_console.py && python3 scripts/guard_claims.py
```

## Verificación de que todo funciona

| Comprobación | Comando | Resultado esperado |
|---|---|---|
| El proceso vive | `curl -s http://127.0.0.1:8080/healthz` | `{"status":"ok",...}` |
| El almacenamiento responde | `curl -s http://127.0.0.1:8080/readyz` | `{"status":"ready",...}` |
| El token funciona | `curl -s -H "Authorization: Bearer $ROOTCAUSE_API_TOKEN" http://127.0.0.1:8080/api/v1/status` | JSON con `posture` y `hardening` |
| Sin token se rechaza | `curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:8080/api/v1/status` | `401` |
| El catálogo se publica | `cargo run -p rootcause-server -- rules` | 18 reglas listadas |
| La consola carga | Abrir `http://127.0.0.1:8080` | Panel de defensa, con diálogo de token |

## Errores frecuentes durante la instalación

| Mensaje o síntoma | Causa | Solución |
|---|---|---|
| `falta ROOTCAUSE_API_TOKEN; genera uno con 'rootcause-server token'` | Se arrancó `serve` sin token y sin `--insecure-dev-mode` | Generar el token y exportarlo |
| `ROOTCAUSE_API_TOKEN debe contener al menos 32 caracteres` | Token demasiado corto | Usar el que genera `rootcause-server token` |
| `ROOTCAUSE_API_TOKEN no debe empezar ni terminar con espacios` | Espacio al copiar y pegar | Recortar el valor |
| `el modo de desarrollo inseguro solo puede escuchar en una dirección de loopback` | `--insecure-dev-mode` con `--bind 0.0.0.0:...` | Usar token, o volver a loopback |
| `--trust-forwarded-for solo tiene sentido detrás de un proxy inverso` | Opción activada escuchando en loopback | Quitar la opción, o enlazar fuera de loopback tras el proxy |
| `el almacenamiento 0.2 requiere una URL sqlite:` | Se pasó una URL de PostgreSQL o MySQL | Solo SQLite en esta versión |
| `no se pudo enlazar RootCause Server a 127.0.0.1:8080` | Puerto ocupado | Cambiar `--bind` o liberar el puerto |
| `no se enviará un token por HTTP remoto` (agente) | URL `http://` no local | Usar `https://`, o `--allow-insecure-http` de forma deliberada |
| `registra el activo antes de enviar telemetría` | Telemetría de un `agent_id` desconocido | El agente registra antes de enviar; revisar que ambas llamadas usen el mismo token |
| El agente arranca pero la superficie llega vacía | Falta `ss`, `journalctl` o equivalentes, o faltan privilegios | Revisar las advertencias de la respuesta: llegan como brechas, no como ceros |
| Compilación falla por el enlazador en Windows | Falta el toolchain de C++ de MSVC | Instalar las *Build Tools* de Visual Studio, o compilar en WSL |

## Errores frecuentes de la primera ejecución

- **La consola pide el token una y otra vez.** El token vive en `sessionStorage`: se pierde al cerrar la pestaña. Es intencional.
- **La consola muestra un aviso amarillo permanente.** Significa que `hardening` declara algo que debilita la postura: sin autenticación, o escuchando fuera de loopback. Es informativo y honesto, no un fallo.
- **Aparece un hallazgo de reloj desincronizado en cuanto se enrola un equipo.** La tolerancia por omisión es de 120 segundos entre el instante de observación del agente y el de recepción del servidor. Sincronizar el reloj del equipo, o ajustar `clock_skew_seconds` en la política.
- **El primer ciclo no detecta cambios de archivos.** Correcto: la primera huella *crea* la línea base. El cambio se detecta a partir del segundo ciclo.

---

**Anterior:** 01 · Descripción general · **Siguiente:** 03 · Arquitectura · **Índice:** README

---